CHEAT SHEET · JS INTERVIEW #08 @specsycoder

"5" + 3 vs "5" - 3

Type coercion, the hidden conversion

coercion.js

```
1  "5" + 3; // "53"
2  "5" - 3; // 2
```

output "53" 2

Why it happens

01 + has two jobs

It adds numbers AND joins strings.

02 String wins

If either side of + is a string, it joins: "5" + 3 = "53".

03 - * / % only do math

They convert strings to numbers first: "5" - 3 = 2.

04 Type coercion

JavaScript changes types automatically, without telling you.

The real bug (and the fix)

checkout.js

```
1  // input values are ALWAYS strings
2  price + tax; // "10050"
3  Number(price) + Number(tax); // 150
```

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

Guess the output

Classic coercion questions

expression	result	why
`"5" * "2"`	10	* converts both to numbers
`1 + 2 + "3"`	"33"	left to right: 3, then joins "3"
`"1" + 2 + 3`	"123"	string first, so everything joins
`"53" - 3`	50	- converts "53" to 53
`true + 1`	2	true becomes 1

converter	result
`Number("42")`	42
`Number("")`	0 (not NaN!)
`Number("42px")`	NaN
`parseInt("42px", 10)`	42
`+"5"`	5 (unary plus)

Traps to mention

- **An empty input becomes 0** with Number(""), so validate before you calculate.
- **For <input type="number">,** use `input.valueAsNumber` to get a real number.

Your 20-second answer

"The + operator adds numbers but joins strings, so if either side is a string, '5' + 3 becomes '53'. Minus only does math, so JavaScript coerces '5' to 5 and gives 2. Form inputs are strings, so I convert with Number() before adding."